

Introduction

Welcome to learning programming with the Romi robot! In this course, you'll learn how to program robots for fun or for competition, and how to write software, which you'll be able to do for robots, and for other uses as well. This course assumes no prior experience of writing software. If you have some prior experience and know the basics, you will still find value in this course as it relates to programming robots and the Romi in particular. If you're familiar with FIRST Robotics and want to learn how to program robots that compete in FIRST competitions, this is the perfect course for you. By the time you finish this course you'll have built up skills to contribute to any team you're a part of and created a foundation for yourself for future learning, leading you on a path to becoming an expert software developer. In this first lesson, we'll focus on setting up the Romi and making it move.

Setting up your robot and getting the first program to run can be the most frustrating part of programming. Fortunately, if you have a Romi in front of you, this has already been completed.

Necessary Materials

To complete this course, you'll need the following things:

- **A Romi robot** (of course.)
- **One or more joysticks/controllers for driving the robot.** This course uses a gamepad-style controller (such as an Xbox controller, Logitech F310, or similar) for the examples, but you can use any controller recognized by your computer, as long as it has at least two joystick axes. The course explains how to account for differences in controllers, although there are some instances where driving may feel uncomfortable if you only have a single joystick.
 - **A supply of AA batteries.** You will likely want rechargeable batteries because you will go through a lot of batteries in this course, and it will be expensive and wasteful if you can't recharge them. The Romi uses six batteries at once and a full charge will last up to a few hours under normal use where you spend more time writing code than driving but can run out faster than that if you spend a lot of time driving. The Romi comes with six batteries, so you can get started on the first couple lesson with those batteries while waiting for a rechargeable kit to arrive, if you purchase a kit online.
- **A computer with wireless internet access.** If you have a computer without wireless access, you can buy a wireless adapter for less than \$20. You need wireless internet to connect to the Romi to deploy code and drive.
- **An internet connection.** In addition to connecting to the Romi, this course has links to external materials you'll need to access. In this lesson, you'll need to download two files which are each about a gigabyte, which can take a few minutes on slower connections. These are the only large downloads in the course.

Setting Up Your Romi

There are two main steps in getting started, both of which are explained in detail below: installing VSCode with the WPILib library, and installing firmware onto your Romi. If you **know** that one or both of these steps have been done for you already, you can skip that step

accordingly. If you aren't sure, then do not skip the step.

Turning your Romi On and Off & the Low Battery Signal

The Romi has two power switches – one is a switch, and one is just a button. To turn it on, you simply have to flip the switch to the “On” position. But to turn it off, you have to flip the switch to “Off” *and then* push the power button. If you don't do this, the Romi will remain on and eventually drain its batteries, so make sure you fully turn it off whenever you're not using it.

When the Romi is low on batteries, it will start playing noises. If you hear it playing noises (you'll know it when you hear it, it's loud and annoying), that means the batteries are low and need to be changed. Change them right away as it will stop functioning correctly if you don't.

Step 1 – Installing VSCode and the WPILib

Programming the Romi is done using a library known as “WPILib”. WPILib is built into an installation of an application called “VSCode”, which is what you will use to write and deploy code to the Romi. There is documentation for the WPILib which includes guides on how to get started. To begin, follow the guide linked below to download VSCode. Note that the VSCode download and the Romi images are 1-2 gigabytes each and may take a few minutes to download.

Installing VSCode THIS HAS BEEN DONE ON THE LAB COMPUTERS. IT IS HERE FOR REFERENCE SO YOU CAN INSTALL IT AT HOME. SKIP TO PART 2.

1. Start by opening this link:

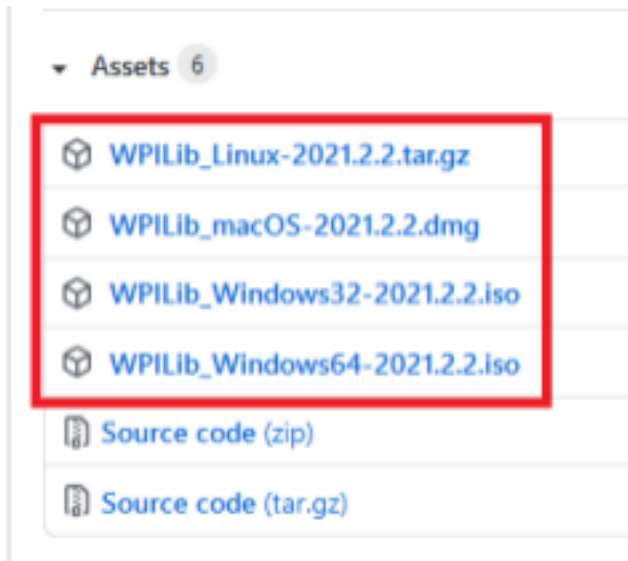
<https://docs.wpilib.org/en/stable/docs/romi-robot/programming-romi.html#programming-the-romi> You will be using **Java**.

2. There will be a link to GitHub which you need to follow:

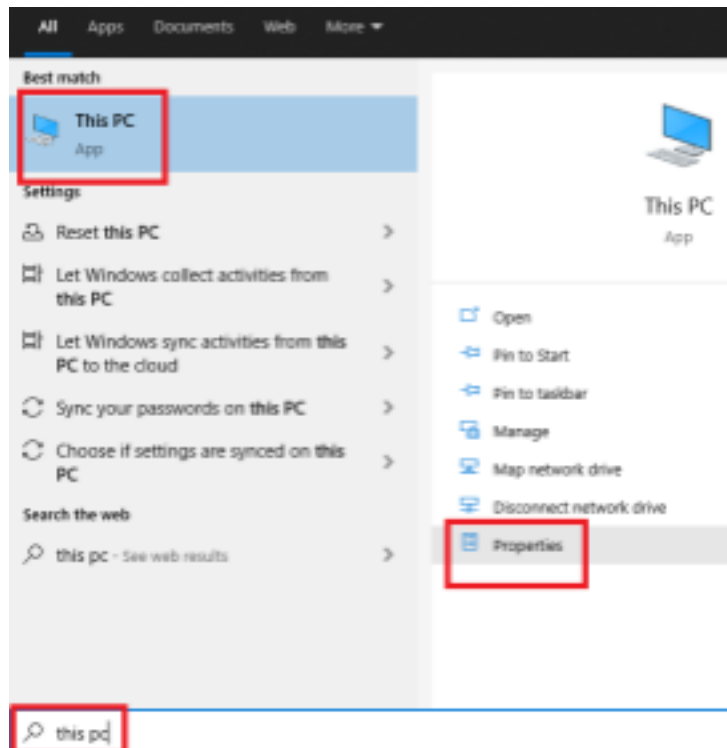
Prerequisites

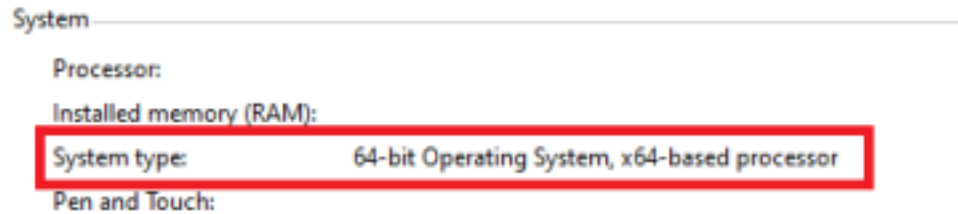
You can download the latest release of the installer from **GitHub**. Ensure that you download the correct binary for your OS and architecture.

3. Once on GitHub, the actual downloads are at the bottom of the page; you'll need to select the proper one based on which operating system you're using (Windows, Mac, or Linux) and whether your system is 32-bit or 64-bit. You don't need to read any of the other text on this page, you just need to find the right download. If you're on Windows and unsure which one to download, look at sub-point a underneath the image below.

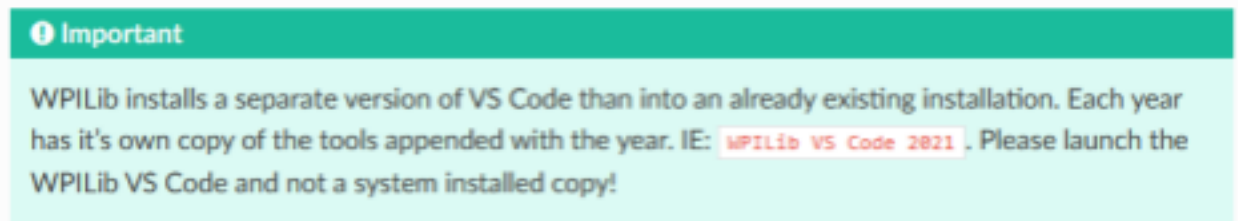


- a. On a Windows computer, if you are unsure of whether it's 32- or 64-bit, press the Windows key (or click on the start menu), type "This PC" or "Computer", and click the "Properties" link that appears in the search menu. Then, in the "System" section, look at the "System type" field, which will tell you if your system is 32- or 64-bit.





4. Once you have the right file downloaded from GitHub, go back to the WPILib documentation using the same link as before:
https://docs.wpilib.org/en/stable/docs/zero-to-robot/step_2/wpilib-setup.html
5. In the documentation, follow all the steps on the page, starting with the “Extracting the Installer” step. There are images to guide you. If you are unsure what options to choose at any point through the setup, follow the default recommendations that the documentation gives you. If you’re unsure about “Install for this User” or “Install for all Users”, you can choose “Install for this User”.
6. Partway through the page you’ll notice a green warning similar to this:



If this is your first time installing VSCode, you probably don’t need to worry about this warning too much, but if for any reason you have a version of VSCode installed your computer already, pay attention to this. Only the new version that you’re installing will work – any prior installation will NOT work, even if it was done for a robotics team.

7. Make sure to read through the page thoroughly while you follow the steps. It is easy to make a mistake, but if you do follow the steps exactly, it will work. If it does not work, you can attempt to backtrack and see what step you missed. Starting over from scratch can sometimes be an easier option as well.

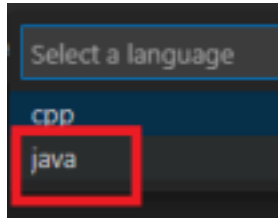
Part 2 - Running Your First Program

Navigate to this page in your browser: [Programming the Romi](https://docs.wpilib.org/en/stable/docs/romi-robot/programming-romi.html)

(<https://docs.wpilib.org/en/stable/docs/romi-robot/programming-romi.html>)

and open VSCode. Make sure you’re connected to your Romi through your wifi network.

Follow the steps in the linked page all the way to the end. There are a couple items that you’ll need to specify that are not covered by the linked page. First, when it asks you to select a language, pick Java:



For the “Base Folder” field, you can choose whatever you want in your filesystem, but it might be a good idea to create a folder called “Development” in an easy-accessible location, where you put all your Romi projects. For the project name, you can again enter whatever you want. “ExampleProgram” might be a good name for your first project. Leave the “Create a new folder?” checkbox checked. For “Team Number”, if you belong to an FRC team, you can enter your team’s number. If you do not, or you don’t know what an FRC team is, you can enter “0000”. Do not check the “Enable Desktop Support” checkbox. Here’s an example:



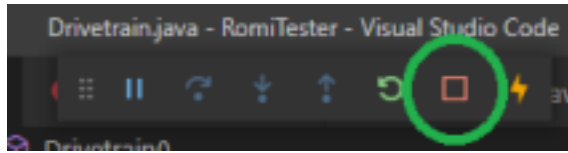
The image shows a dark-themed web interface for creating a new project. At the top left is a logo consisting of a red gear-like hexagon with a white 'W' inside. Below the logo, the title 'Welcome to WPI Lib New Project Creator' is displayed in a large, bold, white font. Underneath the title, there are three buttons: 'example', 'java', and 'RomiReference'. The 'example' button is highlighted. Below these buttons, the text 'Base Folder. Select a base folder to place the new project into.' is shown. A text input field contains the path 'c:\Users\yourname\Documents\Development\Romi Lessons\Projects\Lesson 1'. Below this, there is a button labeled 'Select a new project folder'. The next section is 'Project Name', with a text input field containing 'ExampleProgram'. Below that, the text 'Create a new folder?' is followed by a checked checkbox. A note below says 'This creates a new folder at Base Folder\Project Name. Highly recommended to be checked'. The 'Team Number' section has a text input field with '0000'. The 'Enable Desktop Support' section has an unchecked checkbox. A note below says 'This is needed for simulation and unit testing support, however there are some cases where time.' At the bottom, there is a button labeled 'Generate Project'.

Back on the “Programming the Romi” webpage, you can ignore the paragraph right near the end that says “If you changed the Romi network settings...”. This means the last thing you’ll do in the instructions is press F5, which will deploy the example project. Go ahead and do this!

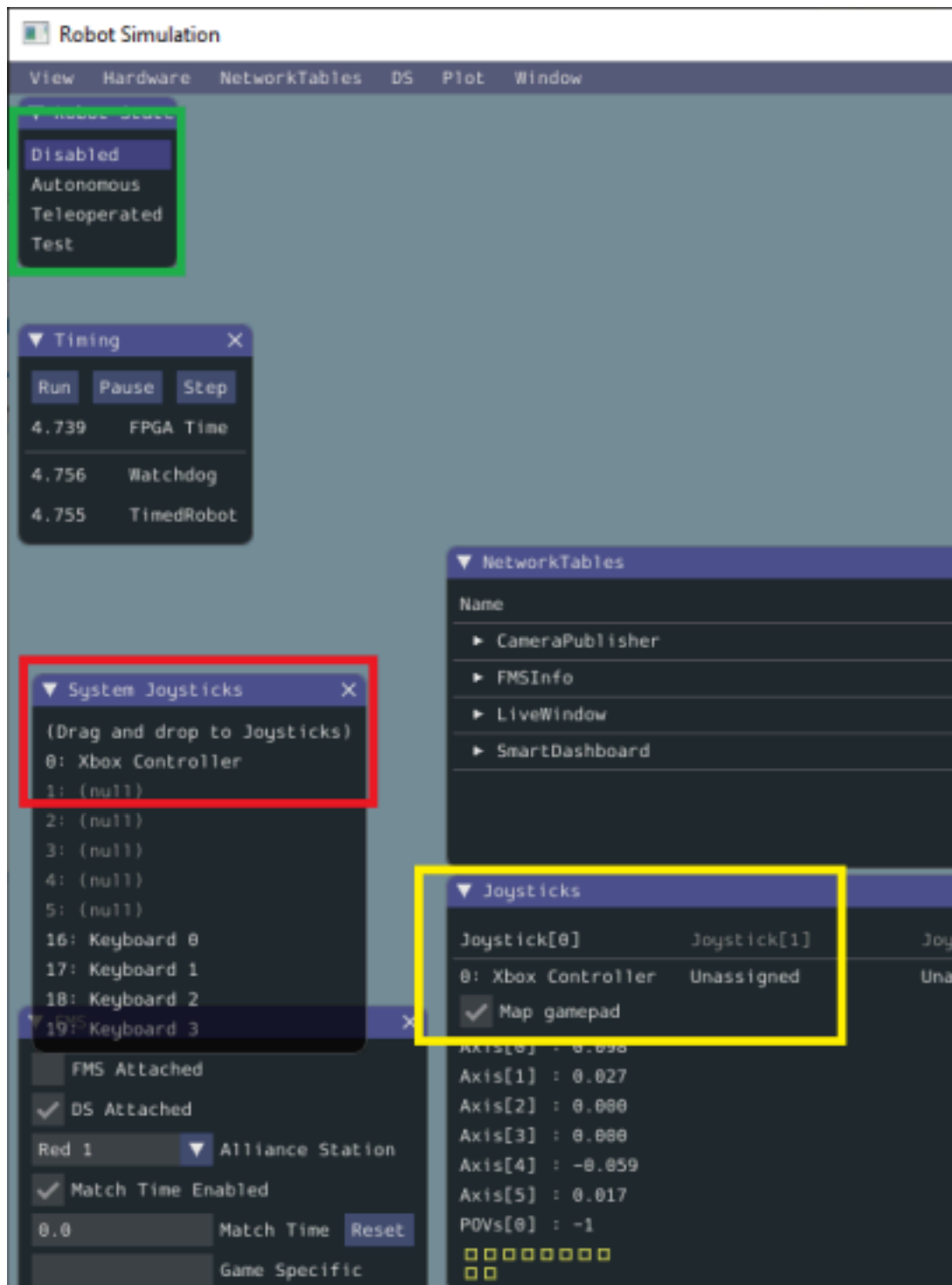
If when you press F5 you get any sort of build error (you will see red text near the bottom of your screen saying build failed), try rebooting VSCode *and* your Romi, re-connecting to your Romi’s wifi (remember that will take a minute for the wifi network to pop up), and deploying again. Sometimes VSCode will fail to contact the Romi even if there is nothing wrong with your code, but rebooting solves this.

After you deploy your code, VSCode will bring up a new interface (called the “Robot Simulation” interface) that looks something like this. **Note 1:** Sometimes the Robot Simulation window will not open the first time you press F5. If this happens to you, simply click the red

square to end your program, and press F5 again.



Note 2: If you have the FRC Driver Station application installed on your computer, you may not see the Robot State box in the upper left of the image below. Instead, you will use the FRC Driver Station app to enable/disable your robot and select modes. If you don't know what the FRC Driver Station is, don't worry about this note.



This interface lets you control the Romi and see various output from the onboard sensors. But what we're most interested in right now is making it move. The simplest way to do this with the sample

program is to enable its autonomous program. Enabling the autonomous program will cause the Romi to drive forward about ten inches, turn around, drive back to its original location, and then turn around again. Make sure you have the Romi somewhere where it can safely do this (not at

the edge of a desk or table) and be ready to enable it. Look at the photo above, and find the “Robot State” box, in the upper left, highlighted in green. Here you will see a list of four robot states – Disabled, Autonomous, Teleoperated, and Test. *As soon as you click Autonomous, the Romi will enable and immediately begin its autonomous routine.* When you’re ready, give it a try! After the Romi finishes moving, click the Disabled option to disable it again. Although the Romi is lightweight and low power, making sure robots are disabled when not in direct use is a critical safety practice.

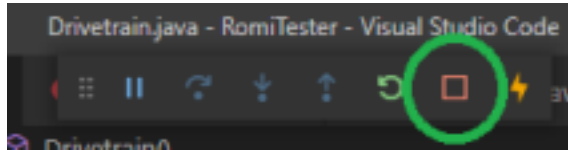
Now that you’ve made the Romi move on its own, it’s time to try driving it yourself. If you look at the window called “System Joysticks”, highlighted in red, you will see any joysticks that you have plugged in to your computer. In the example, there is an Xbox controller plugged in, but you can use anything that your computer recognizes as a joystick. There’s an additional window called simply “Joysticks”, highlighted in yellow, and these are the joysticks that your code and Romi are looking at for input. You may not see any joysticks listed in this window. If that is the case, you can simply drag and drop your joysticks from the System Joysticks window, to the Joysticks window. In the example photo, the Xbox Controller has been dragged to joystick 0. Do likewise – drag whatever joystick you want to use as your controller to the 0 slot. If you have multiple joysticks plugged in to your computer, you may need to do some quick tests to figure out which one is which. How to do this is explained next.

After dragging your joystick to the Joysticks window, try moving your joystick around and pressing some buttons. You will see the yellow squares at the bottom of the Joysticks window fill/unfill as you press and release buttons, and you can see the axis values change in value as you move the sticks around. This is how you can test which joystick in the System Joysticks is which if you are not sure. If you have one joystick dragged to the joysticks window and pressing buttons/moving the sticks doesn’t change any of the values, you probably dragged a different stick.

Once you’ve gotten your joystick to register, and you’ve confirmed it’s in the Joystick[0] slot, you’re ready to enable your robot! Look back at the robot state box, and when you’re ready, click Teleoperated. This will enable the robot. Now you can start trying to drive it around! If you press various buttons and/or sticks on your joystick/controller, you should be able to make the robot move. However, it may or may not control as you expect. For example, by default, if you’re using an Xbox controller, the left joystick will control the robot moving forward and backward, but the *left trigger* will control turning, and you’ll only be able to turn it one direction since the left trigger can only register positive values (as opposed to a joystick, which can register positive or negative values, e.g. if you push it forward or backward.) Now disable the robot. Your first task for you to start programming will be to fix the controls so it can drive normally. If your robot *does* drive as expected, which could happen for some controllers, then you can still follow the lesson. You’ll simply change it to drive abnormally, and then fix it again.

Ending the Executing Program

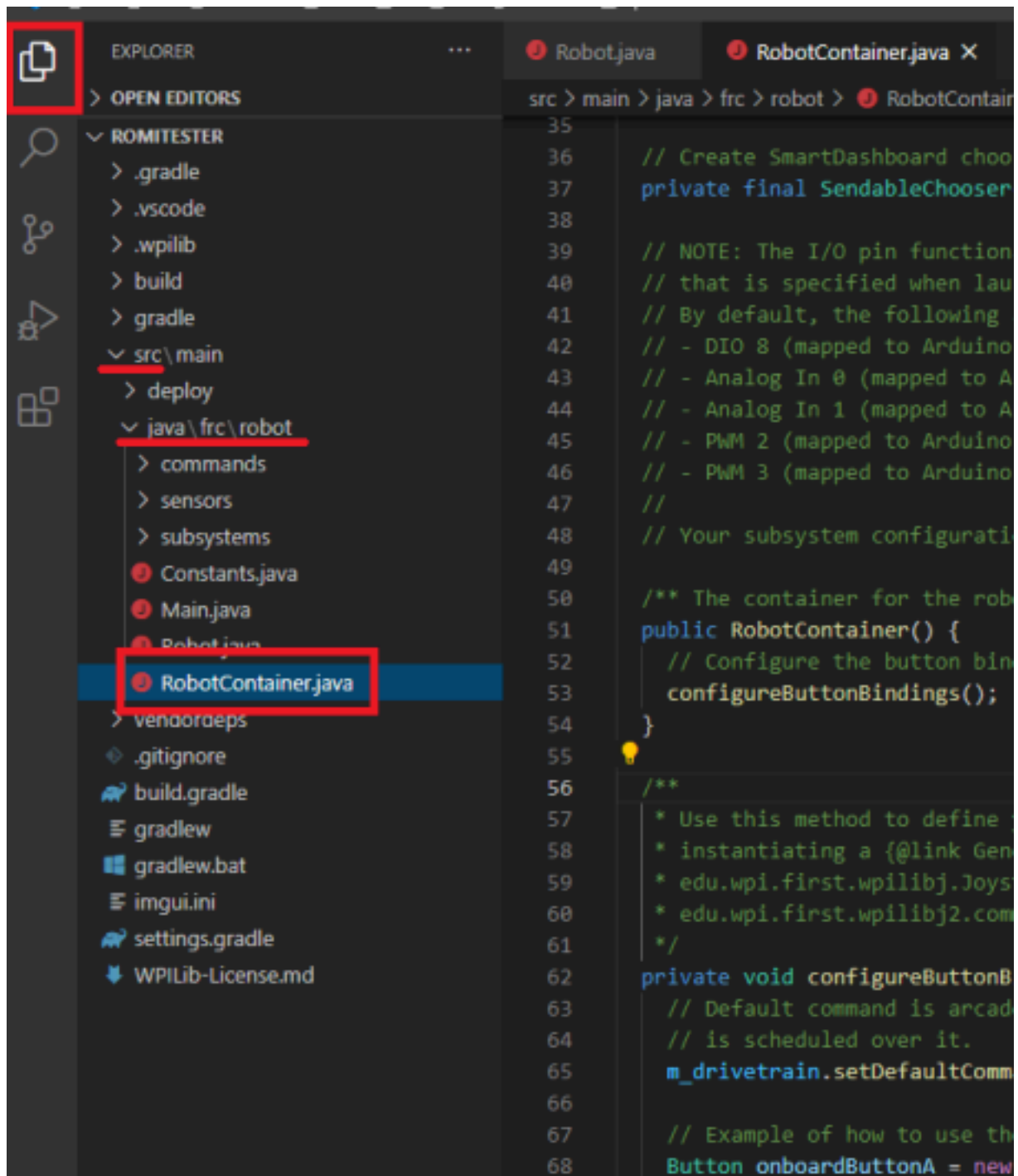
After you press deploy your code, a small box with a few buttons will pop up in the top of VSCode. While this box is visible, you may not be able to deploy additional code to the Romi – VSCode may give you an error. When you want to deploy new code, simply click the red square in this box first, and then you can deploy your code:



Fixing the Controls

As discussed above, you can look at the robot simulation interface, and see changes in the joystick section as you move the joysticks or press buttons. Decide which joystick axis you want to control turning. For example if you're using an Xbox controller, this might be the left joystick being pushed left or right, or the right joystick being pushed left or right. If you're using a single standalone joystick, you don't have as many options. But either way, decide what you want, and then start pushing the joystick that way. On the joysticks interface you will see a value change. For example, it might move from 0 to 1 or -1 depending on which direction you push it. The important thing here is to note which axis is changing. The axes are labeled from 0 to 5 in brackets, like so: Axis[4]. Determine which axis is changing, and remember this number.

Now let's dig into the code. Back in VSCode, open the "Explorer" (highlighted in red, upper left, see the picture below) and then click on src, java, frc, robot, and then finally double click on RobotContainer.java to open it.



If you're brand new to programming, there is a lot to take in here – but don't worry about that. We'll discuss everything in time, but for now let's get those controls fixed. Scroll down in RobotContainer.java until you see line number 95. The lines are numbered on the left side of the code. On line 95, you'll see this:

At the end of the line, you'll see "m_controller.getRawAxis(2))", highlighted in red above. This line sets which axes of the joystick are being used to control the robot. Change the 2 to whatever axis you identified when testing the controls. If your controls were already correct, try

changing it to something else, following the next step, and then changing it back. Close the Robot Simulation Window. Once you do this, you can press ctrl+s to save, and F5 to deploy your code to the Romi again. Once the code finishes deploying, you can enable it again and try out your new controls. If you put the right axis, you'll be able to drive your robot comfortably! If the controls aren't what you expected, double check your axes again when you push on them using the joysticks interface, and make sure you changed the correct value in the code. Now that you have your Romi driving around, have some fun with it! You've taken your first steps into writing code and you already have a robot driving around.